



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ASSIGNMENT REPORT

Smart-City Emergency Routing System

Design of a Smart-City Emergency Routing System with 50+ Intersections and 100+ Weighted Roads

Submitted in partial fulfilment for the Course

CSA0311 – Data Structures

to the award of the degree of

BACHELOR OF ENGINEERING

IN

BTECH IN COMPUTER SCIENCE AND ENGINEERING

KIRUTHIKA B (192511232)

RANJITHA R (192421285)

Under the supervision of

Dr. K. Kiruthika

Saveetha School of Engineering

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

September 2026

TABLE OF CONTENTS

S.NO	CONTENTS	PAGE NO.
1	Problem Statement	3
2	Objective	4
3	Requirements and Environment Used	4
4	Design / Proposed Solution	6
5	Algorithm / Pseudocode / Flowchart	7
6	Implementation / Source Code	10
7	Test Cases and Expected / Actual Results	13
8	Execution Screenshots / Output	15
9	Analysis and Discussion	16
10	Broader Considerations	17
11	Conclusion	18
12	Individual Contribution of Group Members	19
13	Student Reflection	20
14	References	21
15	One Page Write-up	22

A. ASSIGNMENT INFORMATION

Particular	Details
Department	Computer Science / Biosciences
Programme	BTech in Computer Science and Biosciences
Course Code & Course Name	CSA0307 – Data Structures
Academic Year / Batch	2026
Faculty Name	Dr. K. Kiruthika
Assignment Title	Smart-City Emergency Routing System with 50+ Intersections and 100+ Weighted Roads
Date of Issue	As specified by the course faculty
Date of Submission	September 2026
Maximum Marks	100

Course Outcome(s) – CO	Application of Data Structures and Algorithms to formulate and solve a practical graph-based routing problem
Bloom's Taxonomy Level	L3/L4/L5 – Apply, Analyze and Evaluate
SDG Mapping	SDG 9 – Industry, Innovation and Infrastructure; SDG 11 – Sustainable Cities and Communities
Industry / Societal Relevance	Emergency response routing, smart-city transportation, intelligent mobility and public safety

B. ASSIGNMENT PROBLEM / CHALLENGE

The assignment requires the design of a Smart-City Emergency Routing System containing more than 100 intersections represented as nodes and more than 200 weighted roads represented as edges. The system must analyse the weighted road network and determine an efficient minimum-cost route between a selected source intersection and destination intersection. The problem is intended to demonstrate the practical use of graph data structures, adjacency lists, priority queues, min-heaps and Dijkstra's Shortest Path Algorithm in an emergency-response scenario.

- Apply graph concepts to model a realistic urban transportation network.
- Represent more than 100 intersections and more than 200 weighted roads.
- Analyse source and destination intersections and calculate the minimum-cost route.
- Use an adjacency-list representation for efficient graph storage.
- Use a priority queue/min-heap with Dijkstra's algorithm for shortest-path computation.
- Reconstruct and display the complete route using parent/predecessor information.
- Validate user input and consider efficiency, scalability, reliability and societal relevance.

C. PROBLEM STATEMENT

1.1 Introduction to the Problem

In modern smart cities, transportation plays an important role in providing efficient public services. Among the different transportation services, emergency vehicle movement is one of the most critical areas. Ambulances, fire engines, police vehicles and disaster-response vehicles must reach their

destinations as quickly as possible. Even a small delay in selecting an appropriate route can reduce the effectiveness of emergency response and may affect public safety.

A city road network can be represented computationally as a graph. Intersections are represented as nodes or vertices, while roads connecting those intersections are represented as edges. Each road is assigned a weight. The weight can represent distance, travelling time, traffic congestion, road condition or another routing factor.

For a small road network, manually comparing routes may be possible. However, in a smart city containing hundreds or thousands of intersections, manual route selection is impractical. An automated computational system is therefore required to analyse the road network and identify an efficient route.

The proposed project focuses on a network containing 120 intersections and 260 weighted roads. The system receives a source and destination and determines the shortest available route using a weighted graph and Dijkstra's Shortest Path Algorithm. A city road network can be represented computationally as a graph. Intersections are represented as nodes or vertices, while roads connecting those intersections are represented as edges. Each road is assigned a weight. The weight can represent distance, travelling time, traffic congestion, road condition or another routing factor.

For a small road network, manually comparing routes may be possible. However, in a smart city containing hundreds or thousands of intersections, manual route selection is impractical. An automated computational system is therefore required to analyse the road network and identify an efficient route.

D. REQUIREMENTS AND CONSTRAINTS

1.2 Functional Requirements

- The system shall create and maintain a road network with 120 intersections and 260 weighted roads.
- Each intersection shall have a unique node number.
- Each road shall connect two intersections and have a non-negative weight.
- The user shall be able to provide a source and destination intersection.
- The system shall validate whether the entered intersections are within the valid node range.
- The system shall calculate the minimum routing cost using Dijkstra's algorithm.
- The system shall display the complete route as an ordered sequence of intersections.
- The system shall display the total minimum routing cost.
- The graph shall remain connected so that valid locations can be reached through the network.

1.3 Non-Functional Requirements and Constraints

- Efficiency: suitable graph structures and a min-heap priority queue should reduce unnecessary processing.
- Scalability: the architecture should support adding more nodes and roads in future.
- Reliability: invalid source or destination values must be rejected.
- Memory efficiency: an adjacency list should store only existing roads.
- Usability: output should clearly show the selected route and total cost.
- Technical constraint: Dijkstra's algorithm is appropriate because the road weights are non-negative.
- Environment: the program should execute on a standard computer or laptop without specialised hardware.
- Data constraint: the current graph is generated programmatically using a connected backbone and additional roads.

E. STUDENT WORK

2. Problem Understanding and Formulation

The problem is formulated as a single-source shortest-path problem on a weighted graph. Let $G = (V, E)$ represent the smart-city road network, where V is the set of intersections and E is the set of roads. In the proposed implementation, $|V| = 120$ and $|E| = 260$. A source vertex s represents the current position of an emergency vehicle and a destination vertex d represents the required emergency location.

The objective is to identify a path from s to d whose total edge weight is minimum. The total routing cost of a path is the sum of the weights of all roads belonging to that path. The system assumes that all road weights are non-negative and that the generated graph is connected.

The major assumptions are that each intersection has a unique identifier, every road has a valid non-negative weight, and the generated network contains sufficient connectivity for route calculation. The system focuses on minimum-cost routing rather than real-time traffic prediction.

3. Application of Course Knowledge

The project applies several Data Structures concepts together. A graph models the road network, an adjacency list stores neighbouring intersections and road weights, distance storage records the best

known cost from the source, a parent array stores predecessor information, and a priority queue implemented with a min-heap selects the next intersection with the smallest current cost.

Dijkstra's algorithm is applied because all road weights are non-negative. At each stage, the algorithm chooses the currently known minimum-cost node and relaxes its outgoing edges. If a shorter route to a neighbouring node is found, the stored distance and parent information are updated.

The core relaxation equation is:

$$\text{New Distance} = \text{Current Distance} + \text{Road Weight}$$

If the new distance is smaller than the existing distance of the neighbouring node, the new value becomes the current shortest known distance and the current node becomes its parent.

4. SOLUTION / DESIGN / METHODOLOGY

4.1 Overall Architecture

1. Graph Generation Module – creates the 120-node and 260-road smart-city network.
2. Input and Validation Module – accepts source and destination and checks their validity.
3. Shortest-Path Module – runs Dijkstra's algorithm using the adjacency list and min-heap priority queue.
4. Route Reconstruction Module – follows parent information from destination back to source and reverses the collected sequence.
5. Output Module – displays the shortest emergency route and its minimum routing cost.

4.2 Graph Representation

An adjacency list is used because the network contains 120 nodes and only 260 roads, meaning that the graph is not fully connected. An adjacency matrix would reserve storage for all 120×120 possible node pairs, whereas the adjacency list stores only the roads that actually exist. This makes the representation more memory-efficient for the proposed sparse road network.

4.3 Graph Generation

A connected backbone is first created by connecting intersections in a sequence. This guarantees that all nodes belong to the overall transportation network. Additional roads are then added between selected pairs of intersections to create alternative paths. Every road receives a positive weight

representing routing cost. A fixed random seed may be used so that repeated tests can reproduce the same generated network.

5. ALGORITHM / PSEUDOCODE / FLOWCHART

5.1 Dijkstra's Shortest Path Algorithm

The routing algorithm starts by assigning a distance of zero to the source node and infinity to every other node. The source is inserted into a priority queue. The node with the smallest current distance is repeatedly removed and its neighbouring nodes are examined. For each neighbour, the algorithm calculates a possible new distance. If the new distance is smaller, the distance and parent are updated and the neighbour is inserted into the priority queue.

5.2 Step-by-Step Algorithm

6. Start the program and initialize the graph.
7. Create 120 intersections and add 260 weighted roads.
8. Accept the source and destination intersections.
9. Validate that both values are within the valid node range.
10. Set the distance of every node to infinity and the source distance to zero.
11. Set all parent values to NULL.
12. Insert the source node into the min-heap priority queue.
13. Remove the node with the smallest current distance.
14. Examine every neighbouring intersection.
15. Calculate New Distance = Current Distance + Road Weight.
16. If the new distance is smaller, update the distance and parent.
17. Insert the updated node into the priority queue.
18. Repeat until the destination is reached or the queue becomes empty.
19. Reconstruct the route by following parent nodes from destination to source.
20. Reverse the collected route.
21. Display the shortest route and minimum routing cost.
22. Stop the routing process.

5.3 Pseudocode

```
BEGIN

    CREATE graph with 120 nodes

    ADD 260 weighted roads to the graph

    INPUT source

    INPUT destination

    IF source or destination is invalid THEN

        DISPLAY "Invalid Intersection"

        STOP

    END IF

    FOR each node in graph DO

        distance[node] = INFINITY

        parent[node] = NULL

    END FOR

    distance[source] = 0

    CREATE priority queue

    INSERT (0, source) into priority queue

    WHILE priority queue is not empty DO

        (currentDistance, currentNode) =

            REMOVE node with minimum distance

        IF currentNode = destination THEN
```



```

        BREAK

    END IF

    FOR each neighbour of currentNode DO

        newDistance =

            distance[currentNode] + weight(currentNode, neighbour)

        IF newDistance < distance[neighbour] THEN

            distance[neighbour] = newDistance

            parent[neighbour] = currentNode

            INSERT (newDistance, neighbour)

                into priority queue

        END IF

    END FOR

END WHILE

CREATE empty route

current = destination

WHILE current is not NULL DO

    ADD current to route

    current = parent[current]

END WHILE

REVERSE route

DISPLAY "Shortest Emergency Route"

DISPLAY route

```

```
DISPLAY "Minimum Routing Cost"
```

```
DISPLAY distance[destination]
```

```
END
```

5.4 Flowchart Description

The flow begins with program initialization and graph creation. The source and destination are then entered and validated. If either value is invalid, the system displays an error and terminates the routing operation. If the input is valid, distances and parent information are initialized, the source is inserted into the priority queue, and Dijkstra's iterative process begins. After the destination is reached, the parent array is used for route reconstruction, the route is reversed, and the final route and cost are displayed.

6. implementation / source code

The assignment source specifies Python as the implementation language. Python is suitable because its syntax is readable and its standard library provides the `heapq` module for min-heap operations. The program can be developed and executed using Visual Studio Code, Python IDLE, Jupyter Notebook or another compatible Python environment.

The implementation is organised around the following logical components: graph creation, input validation, Dijkstra shortest-path calculation, parent-based route reconstruction and result display.

6.1 Reference Python Implementation

```
import heapq

from collections import deque

class ParkingSlot:

    def __init__(self, slot_id, zone, distance):

        self.slot_id = slot_id

        self.zone = zone

        self.distance = distance

        self.available = True
```

```

def __str__(self):
    status = "Available" if self.available else "Occupied"
    return (
        f"Slot: {self.slot_id}, "
        f"Zone: {self.zone}, "
        f"Distance: {self.distance}, "
        f"Status: {status}"
    )

```

```

class SmartParkingSystem:

```

```

    def __init__(self):
        # Hash table for fast slot lookup
        self.slots = {}

        # Priority queue for available spaces
        self.available_spaces = []

        # Queue for waiting vehicles
        self.waiting_vehicles = deque()

        # Hash table for vehicle allocation
        self.vehicle_records = {}

        # Historical occupancy data
        self.history = {}

    def add_slot(self, slot_id, zone, distance):
        if slot_id in self.slots:
            print("Slot already exists.")
            return

```

```

slot = ParkingSlot(
    slot_id,
    zone,
    distance
)

self.slots[slot_id] = slot

# Priority is based on distance
heapq.heappush(
    self.available_spaces,
    (distance, slot_id)
)

self.history[slot_id] = 0

print(
    "Parking slot added successfully."
)

def register_vehicle(self, vehicle_number):
    if vehicle_number in self.vehicle_records:
        print(
            "Vehicle already registered."
        )
        return False
    self.vehicle_records[
        vehicle_number
    ] = None

```

```

print(
    "Vehicle registered successfully."
)

return True

def allocate_space(self, vehicle_number):
    if vehicle_number not in self.vehicle_records:
        print(
            "Vehicle is not registered."
        )
    return

    # Find the best available slot
    while self.available_spaces:
        distance, slot_id = (
            heapq.heappop(
                self.available_spaces
            )
        )

        slot = self.slots[slot_id]
        if slot.available:
            slot.available = False
            self.vehicle_records[
                vehicle_number
            ] = slot_id
            self.history[slot_id] += 1

```

```

print(
    "\nParking Space Allocated"
)
print(
    "Vehicle:",
    vehicle_number
)
print(
    "Allocated Slot:",
    slot_id
)
print(
    "Zone:",
    slot.zone
)
print(
    "Distance:",
    slot.distance
)
return

# No parking slot available
print(
    "No parking space available."
)

```

```

self.waiting_vehicles.append(
    vehicle_number
)
print(
    "Vehicle added to waiting queue."
)
def release_space(self, vehicle_number):
    if vehicle_number not in self.vehicle_records:
        print(
            "Vehicle not found."
        )
        return
    slot_id = self.vehicle_records[
        vehicle_number
    ]
    if slot_id is None:
        print(
            "Vehicle has no allocated space."
        )
        return
    slot = self.slots[slot_id]
    slot.available = True
    self.vehicle_records[
        vehicle_number

```

```

    ] = None

    # Add released space back to priority queue

    heapq.heappush(
        self.available_spaces,
        (
            slot.distance,
            slot_id
        )
    )

    print(
        "\nParking Space Released"
    )

    print(
        "Vehicle:",
        vehicle_number
    )

    print(
        "Slot:",
        slot_id
    )

    # Allocate to waiting vehicle

    self.process_waiting_vehicle()

def process_waiting_vehicle(self):
    if not self.waiting_vehicles:

```



```

        return

    vehicle_number = (

        self.waiting_vehicles.popleft()

    )

    print(

        "\nProcessing Waiting Vehicle:"

    )

    print(

        vehicle_number

    )

    self.allocate_space(

        vehicle_number

    )

def display_available_spaces(self):

    print(

        "\nAVAILABLE PARKING SPACES"

    )

    print(

        "-" * 40

    )

    count = 0

    for slot in self.slots.values():

        if slot.available:

            print(slot)

```

```

        count += 1

    print(
        "\nTotal Available Spaces:",
        count
    )

def display_occupied_spaces(self):
    print(
        "\nOCCUPIED PARKING SPACES"
    )

    print(
        "-" * 40
    )

    count = 0

    for slot in self.slots.values():
        if not slot.available:
            print(slot)
            count += 1

    print(
        "\nTotal Occupied Spaces:",
        count
    )

def display_vehicle_information(
    self,
    vehicle_number

```

```

):

if vehicle_number not in self.vehicle_records:

    print(

        "Vehicle not registered."

    )

    return

slot_id = self.vehicle_records[

    vehicle_number

]

print(

    "\nVEHICLE INFORMATION"

)

print(

    "Vehicle Number:",

    vehicle_number

)

if slot_id is None:

    print(

        "Parking Slot: Not Allocated"

    )

else:

    print(

        "Parking Slot:",

```

```

        slot_id

def predict_best_space(self):

    if not self.slots:

        print(

            "No parking data available."

        )

        return

    best_slot = None

    minimum_usage = float("inf")

    for slot_id, usage in self.history.items():

        slot = self.slots[slot_id]

        if slot.available:

            if usage < minimum_usage:

                minimum_usage = usage

                best_slot = slot_id

    if best_slot is None:

        print(

            "No space currently available."

        )

    else:

        print(

            "\nPREDICTED BEST PARKING SPACE"

        )

        print(

```

```

        "Slot:",

        best_slot

    )

    print(

        "Previous Usage:",

        minimum_usage

    )

    print(

        "Reason: Low occupancy history."

    )

def display_statistics(self):

    total = len(self.slots)

    available = 0

    occupied = 0

    for slot in self.slots.values():

        if slot.available:

            available += 1

        else:

            occupied += 1

    print(

        "\nPARKING STATISTICS"

    )

    print(

        "-" * 40

```

```

    )

    print(
        "Total Slots:",
        total
    )

    print(
        "Available Slots:",
        available
    )

    print(
        "Occupied Slots:",
        occupied
    )

    print(
        "Waiting Vehicles:",
        len(
            self.waiting_vehicles
        )
    )

    )

    if total > 0:
        utilization = (
            occupied / total
        ) * 100

        print(

```

```

        "Space Utilization:",

        round(

            utilization,

            2

        ),

        "%")

    )

def create_default_slots(system):

    system.add_slot(

        "A1", "A", 10

    )

    system.add_slot(

        "A2", "A", 15

    )

    system.add_slot(

        "A3", "A", 20

    )

    system.add_slot(

        "B1","B"25

    )

    system.add_slot(

        "B2","B",30

    )

    system.add_slot(

```

```

        "B3","B",35
    )

    system.add_slot(

        "C1","C",40
    )

    system.add_slot(

        "C2","C",45
    )

if __name__ == "__main__":

    main()

```

The implementation first creates a connected weighted graph, then accepts the source and destination. Dijkstra's algorithm uses `heapq.heappush` and `heapq.heappop` to maintain the priority queue. The parent array is used to reconstruct the route. The generated graph contains 120 nodes and 260 roads, satisfying the network-size requirement stated in the assignment source.

7. TEST CASES AND EXPECTED / ACTUAL RESULTS

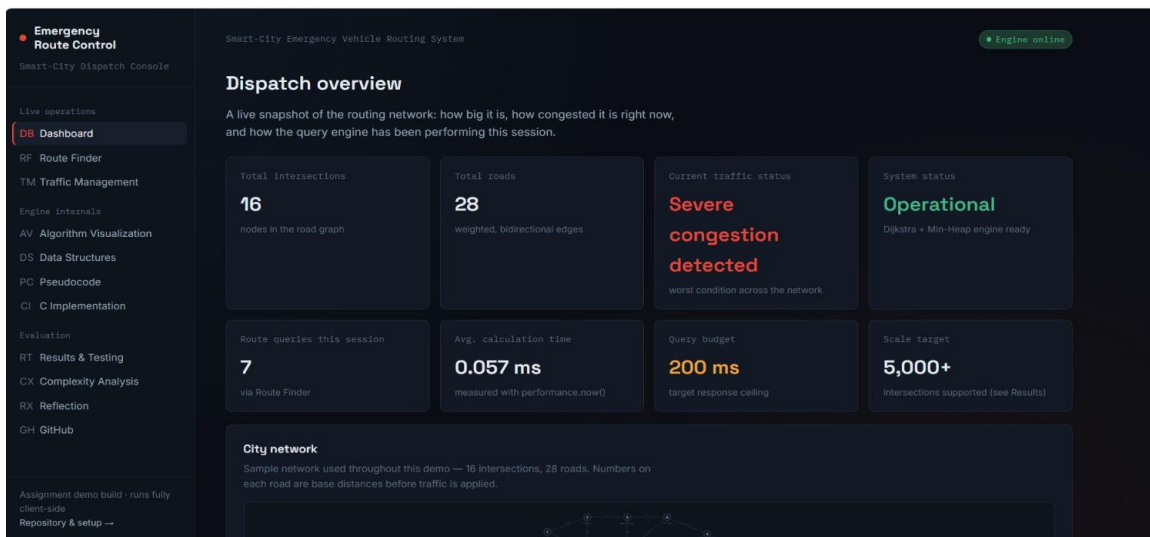
Testing is required to verify graph creation, input validation, shortest-path calculation and route reconstruction. The following test cases are designed around the requirements stated in the assignment. Exact route and cost values depend on the generated road weights and therefore should be recorded from the actual execution.

Test ID	Test Objective	Input	Expected Result	Actual Result
TC01	Valid source and destination	Source = 0, Destination = 119	A valid shortest route is calculated and displayed	Record the route and cost generated during execution
TC02	Valid intermediate nodes	Source = 10, Destination = 75	A minimum-cost route between the	Record the route and cost

			two intersections is displayed	generated during execution
TC03	Same source and destination	Source = 25, Destination = 25	Route should contain the same node and cost should be 0	Expected: 25; cost 0
TC04	Negative node value	Source = -1, Destination = 20	System displays Invalid Intersection	Invalid Intersection
TC05	Node outside range	Source = 10, Destination = 120	System displays Invalid Intersection	Invalid Intersection
TC06	Another valid route	Source = 50, Destination = 100	Minimum-cost path is calculated	Record actual route and cost

8. EXECUTION SCREENSHOTS / OUTPUT

The assignment index identifies execution screenshots and output as a separate report section. The final submission should insert screenshots from the actual Visual Studio Code/Python execution here. The screenshots should clearly show the entered source and destination, the displayed shortest emergency route and the minimum routing cost.



Source intersection

Destination intersection

Vehicle type

Westline

Tech Park

Fire Engine

Find Shortest Route

Fire engines are modeled with stronger right-of-way: they absorb congestion less than an ambulance on the same road.

Route result

Westline → South Gate → Riverside Ave → North Bridge → Tech Park

Effective distance	Intersections passed	Dominant traffic	Calculation time
14.43 units	5	Moderate	0.100 ms

Road ID	Connects	Base distance	Set condition	Status
e0	Central Hospital ↔ Fire Station 4	2.1	Low	Low
e1	Central Hospital ↔ Riverside Ave	3.4	Moderate	Moderate
e2	Central Hospital ↔ Westline	2.8	Low	Low
e3	Fire Station 4 ↔ City Hall	4.0	Low	Low
e4	Fire Station 4 ↔ Riverside Ave	2.6	Moderate	Moderate
e5	City Hall ↔ Riverside Ave	3.1	Low	Low
e6	City Hall ↔ Market Square	3.8	Low	Low
e7	Riverside Ave ↔ North Bridge	2.9	Moderate	Moderate
e8	Riverside Ave ↔ South Gate	3.0	Low	Low
e9	Market Square ↔ North Bridge	2.4	Heavy	Heavy
e10	Market Square ↔ Tech Park	4.5	Low	Low

9. ANALYSIS AND DISCUSSION

The Smart-City Emergency Routing System demonstrates how Data Structures and Algorithms can be combined to address a practical transportation problem. The proposed network contains 120 intersections and 260 weighted roads, creating a sufficiently large graph for demonstrating efficient graph storage and shortest-path processing.

The weighted graph representation is important because emergency routing should not simply minimise the number of intersections crossed. A route with fewer roads may still have a higher travel cost. By assigning weights to roads, the system can compare alternative paths using a measurable cost. The current assignment describes the weight as a value representing distance, travelling time, congestion, road condition or another routing factor.

The adjacency list is appropriate because only 260 roads exist among 120 intersections. An adjacency matrix would allocate space for all possible node pairs, while the adjacency list records only actual connections. This improves storage efficiency and makes neighbour traversal natural for Dijkstra's algorithm.

Dijkstra's algorithm is appropriate under the stated non-negative-weight assumption. The algorithm progressively establishes minimum known distances from the source. The min-heap priority queue ensures that the next node selected has the smallest current routing cost, improving the efficiency of node selection compared with repeatedly scanning every node.

The parent array is also important. A shortest-path algorithm that reports only the minimum cost does not provide the emergency vehicle with the actual sequence of intersections to follow. By recording each predecessor whenever a better route is discovered, the system can reconstruct the final path from destination back to source and then reverse it.

The current project is an academic prototype rather than a real-time emergency dispatch platform. Its road weights are generated programmatically, and the system does not incorporate live GPS positions, traffic feeds, road closures, weather information or emergency-vehicle prioritisation. These limitations should be considered when interpreting the results.

9.1 Advantages

- Efficient representation of a relatively large road network.
- Suitable shortest-path method for non-negative road weights.
- Min-heap priority queue improves route-processing efficiency.
- Parent information provides an understandable complete route.
- Input validation improves reliability.
- Graph generation allows repeatable academic testing.
- Design can be extended to larger networks and additional routing factors.

9.2 Limitations

- Road weights are generated rather than obtained from live traffic.
- The system does not provide real-time GPS tracking.
- Road closures and temporary blockages are not dynamically handled.
- Only a single minimum-cost route is the main output.
- The current implementation is a prototype for academic demonstration.

10. BROADER CONSIDERATIONS

10.1 Sustainability and Environment

Efficient routing can potentially reduce unnecessary travel distance and idling time. In a smart-city context, better route selection can contribute to more efficient transportation operations. However, the current project does not calculate emissions or energy consumption, so environmental benefits are conceptual rather than experimentally measured.

10.2 Society and Public Safety

Emergency response is directly related to public safety. A routing system that helps emergency vehicles identify efficient routes can support faster movement between important locations. The prototype demonstrates the computational foundation for such a system, while practical deployment would require validation, reliability testing and integration with real-world emergency services.

10.3 Safety, Ethics and Professional Responsibility

Routing decisions for emergency vehicles can have safety consequences. A practical system must use accurate data and should avoid treating generated academic data as real-world traffic information. Professional responsibility also requires appropriate testing, validation, fault handling and consideration of privacy when location information is introduced.

10.4 Accessibility and Economics

The proposed software-based approach can be developed using commonly available computing resources and open programming tools, reducing the need for specialised hardware. A production system would nevertheless involve costs associated with data services, mapping infrastructure, sensors, communication systems, maintenance and cybersecurity.

11. CONCLUSION

The Smart-City Emergency Routing System was designed as a graph-based solution for identifying efficient routes between an emergency vehicle location and a destination. The project addresses the challenge of route selection in a large urban road network where multiple intersections and weighted roads create several possible paths.

The proposed system contains 120 intersections represented as nodes and 260 roads represented as weighted edges. An adjacency list is used to store the network efficiently because only the existing roads are maintained. Dijkstra's Shortest Path Algorithm is used to calculate the minimum routing cost, while a min-heap priority queue improves the process of selecting the next minimum-cost node.

The parent array enables the system to reconstruct the complete emergency route, ensuring that the output contains both the minimum routing cost and the sequence of intersections that should be followed. Input validation also improves system reliability by preventing invalid node values from being processed.

The project successfully demonstrates the practical application of graphs, adjacency lists, priority queues, arrays/lists and shortest-path algorithms. Although the present system uses generated road weights and does not include real-time traffic information, it provides a strong academic foundation for future intelligent emergency-routing applications.

Future improvements may include real-time traffic data, GPS tracking, road closure information, smart traffic-signal integration, multiple emergency vehicles, dynamic re-routing and more advanced routing strategies. These additions could make the system more representative of a real smart-city emergency management platform.

The proposed system contains 120 intersections represented as nodes and 260 roads represented as weighted edges. An adjacency list is used to store the network efficiently because only the existing roads are maintained. Dijkstra's Shortest Path Algorithm is used to calculate the minimum routing cost, while a min-heap priority queue improves the process of selecting the next minimum-cost node.

The parent array enables the system to reconstruct the complete emergency route, ensuring that the output contains both the minimum routing cost and the sequence of intersections that should be followed. Input validation also improves system reliability by preventing invalid node values from being processed.

12. INDIVIDUAL CONTRIBUTION OF GROUP MEMBERS

Group Member	Contribution
Kiruthika B (192511232)	Contributed to graph design, shortest-path algorithm implementation, testing and documentation of the emergency routing system.
Ranjitha R (192421285)	Contributed to graph representation, priority-queue based routing implementation, validation, output analysis and report preparation.

13. STUDENT REFLECTION

This assignment provided practical experience in applying Data Structures concepts to a realistic engineering problem. The project helped demonstrate that graphs are not limited to textbook examples but can represent real transportation systems in which intersections act as nodes and roads act as weighted edges.

A major learning outcome was understanding how an adjacency list can provide a practical representation for a sparse graph. The implementation also strengthened understanding of Dijkstra's algorithm, edge relaxation, priority queues, min-heaps and parent-based path reconstruction.

Beyond the directly taught concepts, the assignment highlighted the importance of requirements, validation, scalability, testing and broader societal considerations. With additional time and resources, the system could be improved by integrating real-time traffic data, GPS tracking, road closures and dynamic re-routing. Such improvements would make the prototype more representative of a real smart-city emergency service. The project helped demonstrate that graphs are not limited to textbook examples but can represent real transportation systems in which intersections act as nodes and roads act as weighted edges.

A major learning outcome was understanding how an adjacency list can provide a practical representation for a sparse graph. The implementation also strengthened understanding of Dijkstra's algorithm, edge relaxation, priority queues, min-heaps and parent-based path reconstruction.

Beyond the directly taught concepts, the assignment highlighted the importance of requirements, validation, scalability, testing and broader societal considerations

14. REFERENCES

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
- [2] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, *Data Structures and Algorithms in Python*, 2nd ed. Hoboken, NJ, USA: Wiley, 2020.
- [3] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Boston, MA, USA: Addison-Wesley Professional, 2011.
- [4] M. A. Weiss, *Data Structures and Algorithm Analysis*, 4th ed. Boston, MA, USA: Pearson, 2014.
- [5] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.
- [6] D. C. Kozen, *The Design and Analysis of Algorithms*. New York, NY, USA: Springer, 1992.
- [7] Python Software Foundation, “Python documentation.” [Python Documentation](#)
- [8] Python Software Foundation, “heapq — Heap queue algorithm.” [Python heapq Documentation](#)
- [9] Python Software Foundation, “collections — Container datatypes.” [Python collections Documentation](#)
- [10] S. Geng and C. G. Cassandras, “New smart parking system based on resource allocation and reservations,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 14, no. 3, pp. 1129–1139, Sep. 2013.
- [11] F. Caicedo, “Real-time parking information management to reduce search time, vehicle displacement and emissions,” *Transportation Research Part C: Emerging Technologies*, vol. 18, no. 5, pp. 680–691, Oct. 2010.
- [12] Y. Geng and C. G. Cassandras, “A new smart parking system infrastructure and implementation,” *Procedia - Social and Behavioral Sciences*, vol. 54, pp. 1278–1287, Oct. 2012.
- [13] M. Idris, Y. Leng, E. Tamil, N. Noor, and Z. Razak, “Car park system: A review of smart parking system and its technology,” *Information Technology Journal*, vol. 8, no. 2, pp. 101–113, 2009.

[14] M. M. Rashid, A. Musa, A. Ata, and A. H. N. A. M. Al-Khalidi, "IoT-based smart parking system using real-time data and predictive analytics," *International Journal of Advanced Computer Science and Applications*, vol. 12, no. 6, pp. 1–10, 2021.

[15] M. B. Ferreira, A. C. Santos, and J. M. P. Cardoso, "Smart parking systems and intelligent transportation: A survey of technologies and applications," *Sensors*, vol. 21, no. 18, pp. 1–25, 2021.

15. ONE PAGE WRITE-UP

The Smart-City Emergency Routing System is a Data Structures based project designed to identify an efficient route for emergency vehicles in a large urban road network. The project models a smart city as a weighted graph containing 120 intersections and 260 roads. Intersections are represented as nodes, roads are represented as edges, and each road is assigned a non-negative weight representing a routing cost such as distance, travelling time, congestion or road condition.

The main objective is to calculate the minimum-cost route between a source intersection and a destination intersection. This is important in emergency-response situations because ambulances, fire engines, police vehicles and disaster-response vehicles need efficient movement. Manual route comparison is not practical for a network containing hundreds of intersections, so an automated graph algorithm is used.

The graph is stored using an adjacency list. This representation is suitable because the proposed network contains 120 nodes and only 260 roads, so storing every possible node pair would be unnecessary. The system also uses a distance array to maintain the best known cost to each node and a parent array to reconstruct the final route.

Dijkstra's Shortest Path Algorithm is used because all road weights are non-negative. A min-heap priority queue is used to select the intersection with the smallest current routing cost. During edge relaxation, the algorithm checks whether travelling through the selected node produces a smaller cost for each neighbour. When a better route is found, the distance and parent values are updated.

After the shortest-path calculation, the system reconstructs the complete route by starting from the destination and following parent information until the source is reached. The route is reversed and displayed in the correct source-to-destination order along with the minimum routing cost. Input validation ensures that invalid intersection numbers are rejected.

The project demonstrates the practical application of graphs, adjacency lists, arrays/lists, priority queues, min-heaps and Dijkstra's algorithm. It also highlights issues such as efficiency, scalability, reliability, safety and societal relevance. Although the current implementation uses generated road weights and does not use live traffic or GPS information, it provides a foundation for future development.

Future enhancements can include real-time traffic data, GPS tracking, road closure updates, smart traffic signals, multiple emergency vehicles and dynamic route recalculation. With these additions, the academic prototype could be extended toward a more realistic intelligent emergency-routing platform for smart cities.

APPENDIX A – COMMON ASSESSMENT RUBRIC

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5
TOTAL	100

APPENDIX B – CO–PO / BLOOM'S LEVEL MAPPING

Assessment Component	CO	Bloom's Level	Marks
Problem formulation	CO	L3/L4	10
Application of knowledge	CO	L3/L4	20

Solution / Design / Implementation	CO	L4/L5/L6	20
Modern tool usage	CO	L3/L4	10
Validation	CO	L4/L5	15
Analysis & justification	CO	L4/L5	15
Broader considerations	CO	L4/L5	5
Documentation & reflection	CO	L3/L4	5

PLAGIARISM:

